

ПРАКТИЧЕСКОЕ ЗАНЯТИЕ №4

ТЕМА: Интеграция физики и системы ввода с визуальной отладкой

ЦЕЛЬ

Реализовать базовую физическую подсистему (коллизии, гравитацию) и систему управления вводом, интегрировать их с существующей ECS-архитектурой, а также обеспечить визуальную отладку физических взаимодействий через RenderAdapter.

ЗАДАЧИ

1. Разработать компоненты Rigidbody (скорость, ускорение, масса, гравитация) и Collider (AABB, сфера).
2. Реализовать систему физики (PhysicsSystem), обрабатывающую:
 - применение гравитации;
 - обнаружение столкновений (AABB vs AABB);
 - разрешение коллизий (простейшее отталкивание).
3. Создать InputManager для абстракции устройств ввода (клавиатура, мышь).
4. Реализовать систему событий для оповещения о коллизиях.
5. Обеспечить debug-отрисовку коллизий (AABB-оболочки) через RenderAdapter.
6. Добавить управление 3D-камерой (передвижение, вращение) с помощью ввода.
7. Продемонстрировать взаимодействие объектов в 3D-пространстве под действием физики.

ГЛОССАРИЙ

Термин	Определение
AABB (Axis-Aligned Bounding Box)	Прямоугольный параллелепипед, стороны которого выровнены по осям координат. Простейшая форма коллайдера.
Коллизия	Столкновение двух или более объектов в игровом пространстве.
Разрешение коллизий	Процесс коррекции позиций и скоростей объектов после обнаружения столкновения.
Гравитация	Постоянная сила, приложенная ко всем объектам с компонентом Rigidbody.
InputManager	Компонент, абстрагирующий опрос устройств ввода и предоставляющий унифицированный интерфейс.

Система событий	Механизм, позволяющий объектам подписываться и реагировать на игровые события (например, столкновение).
Debug-визуализация	Отрисовка вспомогательной геометрии (коллайдеры, векторы) для отладки, невидимая в финальной сборке.

НЕОБХОДИМЫЕ ИНСТРУМЕНТЫ

- **Язык:** C++17/20.
- **Среда разработки:** Visual Studio 2022 / CLion / VS Code.
- **Графический API:** OpenGL 3.3+ / DirectX 11 / Vulkan (через существующий RenderAdapter).
- **Математика:** GLM (рекомендуется) или собственная реализация.
- **Система сборки:** CMake.
- **Система контроля версий:** Git.

КРАТКАЯ ТЕОРЕТИЧЕСКАЯ СПРАВКА

Игровая физика — моделирование физических законов в игровом мире. Включает движение тел под действием сил (гравитация, трение) и обработку столкновений.

Коллизии — обнаружение пересечений геометрических объёмов объектов. Простейший тип — AABB (Axis-Aligned Bounding Box), когда прямоугольные параллелепипеды выровнены по осям координат. Проверка столкновения двух AABB сводится к сравнению проекций на каждую ось.

Гравитация — постоянное ускорение, приложенное к объектам (обычно 9.8 м/с^2 вниз по оси Y). В упрощённой модели может быть реализована как постоянное изменение скорости каждого Rigidbody каждый кадр: $velocity += gravity * deltaTime$

Система ввода должна абстрагировать конкретные устройства. Паттерн «Команда» позволяет связать действия игрока (например, «прыжок») с конкретными клавишами, которые можно переназначать.

Debug-визуализация критически важна при разработке физики. Отрисовка коллайдеров (например, прямоугольников вокруг объектов) позволяет сразу видеть, корректно ли работают обнаружение и разрешение коллизий.

Связь с предыдущими работами:

- ECS предоставляет сущности и компоненты (Transform, MeshRenderer).
- RenderAdapter даёт возможность рисовать не только модели, но и debug-геометрию.
- Игровой цикл с фиксированным шагом (из ПЗ №1) важен для стабильной физики.

ИСХОДНЫЕ ДАННЫЕ (с учётом предыдущих работ)

Имеется работающий прототип движка, выполненный в рамках ПЗ №1–3:

- Класс Application с игровым циклом и точным deltaTime.
- Абстрактный RenderAdapter и конкретная реализация для выбранного графического API. Предпочтительнее API (OpenGL/DirectX/Vulkan).
- ECS-система: World, сущности, компоненты Transform, Tag, MeshRenderer, система RenderSystem.
- ResourceManager с поддержкой загрузки мешей и текстур, возможность отрисовки 3D-моделей.
- Возможность отрисовки нескольких 3D-объектов с текстурами.

Все эти наработки являются фундаментом и будут расширены физическими компонентами и системами.

ЗАДАНИЕ И ПОРЯДОК ВЫПОЛНЕНИЯ

Шаг 1. Создание компонентов Rigidbody и Collider

Создание базовых компонентов для описания физических свойств объектов является первым важным шагом при разработке игрового движка или симуляции. Компонент Rigidbody описывает динамические характеристики объекта (такие как скорость, масса и применение сил), тогда как компонент Collider определяет форму объекта и область возможного столкновения.

Цель шага:

Создать базовые компоненты, позволяющие задать физическую природу игровых объектов, включающие следующие элементы:

- **Rigidbody:** Определяет динамику объекта, включая его массу, текущую скорость и способность реагировать на внешнюю среду (например, силу тяжести).
- **Collider:** Определяет геометрические формы объектов и области возможных столкновений.

Подробное описание шагов:

1.1. Компонент Rigidbody

Данный компонент хранит ключевые физические параметры объекта, такие как:

- Текущую скорость (velocity),
- Ускорение (acceleration),
- Массу (mass),
- Флаг применения гравитации (useGravity).

Кроме того, предусмотрена возможность расширенного использования компонента путём добавления дополнительной динамики, хотя пока акцент сделан именно на статичные аспекты.

```
#include <glm/vec3.hpp>
```

```
struct RigidbodyComponent {  
    glm::vec3 velocity;    // Скорость объекта  
    glm::vec3 acceleration; // Ускорение (не активно на данном этапе)  
    float mass;            // Масса объекта  
    bool useGravity;       // Использовать гравитацию?  
};
```

Каждый объект, которому нужен физический отклик на окружающее пространство, получит экземпляр этого компонента. Пока речь идёт исключительно о самой минимальной поддержке динамики, но в будущем сюда можно включить более продвинутые вещи вроде момента инерции и углового вращения.

1.2. Компонент Collider

Коллайдеры необходимы для вычисления областей потенциальных столкновений. Изначально реализуем самый простой вид коллайдеров — Axis Aligned Bounding Box (AABB). Этот компонент хранит размеры ограничивающего прямоугольника и смещение относительно точки трансформации объекта.

Структура выглядит следующим образом:

```
#include <glm/vec3.hpp>
```

```
enum class ColliderType {  
    BOX_COLLIDER,  
    SPHERE_COLLIDER  
};
```

```
struct ColliderComponent {  
    ColliderType type;    // Тип коллайдера (BOX/Sphere)  
    glm::vec3 halfExtents; // Размеры половины коробки или радиус сферы  
    glm::vec3 offset;     // Смещение относительно центра Transform  
};
```

Здесь `halfExtents` задаёт половину размера ограничивающего объёма для коробочного коллайдера, либо радиус для сферического коллайдера. Поле `offset` учитывает возможное смещение коллайдера относительно локального пространства объекта.

Интеграция компонентов в игровую сцену:

Чтобы задействовать созданные компоненты, нужно связать их с каждым объектом, который требует физической модели поведения. Обычно это делается с использованием паттерна "компонентной системы":

```
struct Entity {  
    TransformComponent transform;  
    RigidbodyComponent rigidbody;  
    ColliderComponent collider;  
};
```

Теперь любая сцена может содержать коллекцию таких объектов, а соответствующая физическая система (см. следующий шаг) займётся применением законов физики и обработкой столкновений.

Итог реализации:

Мы создали минимальный набор компонентов, необходимых для описания основных физических характеристик объектов:

- Компонент `Rigidbody`: задаёт динамику объекта (масса, скорость, реакция на внешние воздействия).
- Компонент `Collider`: определяет геометрию объекта и зону потенциального столкновения.

Эти компоненты являются фундаментом будущей физической системы, позволяя построить основы реалистичного поведения объектов в виртуальном мире.

Шаг 2. Создание `PhysicsSystem`

`PhysicsSystem` является одним из ключевых элементов большинства игровых движков и симуляторов. Его задача — отслеживать физическое поведение объектов, применяя законы механики, включая силу тяжести, импульсы, столкновения и взаимодействие объектов.

Основной целью этого шага является создание достаточно простой физической системы, обеспечивающей реалистичное падение объектов под воздействием силы тяжести и предотвращающую проникновение одного объекта внутрь другого.

Задача:

Реализовать физический движатель, который способен:

1. Применять гравитацию к объектам, имеющим компоненты `Rigidbody`.
2. Обновлять позиции объектов на каждом кадре.
3. Выполнять проверку на столкновения и решать конфликты позиций при пересечении объектов.

2.1. Применение гравитации

Каждой итерации обновления физика должна применять ускорение свободного падения к каждому объекту, имеющему компонент `Rigidbody`. Если флаг `useGravity` установлен в значение `true`, к вертикальной составляющей вектора скорости добавляется уменьшение значения, пропорциональное силе тяжести и прошедшему времени (`dt`).

```
for(Entity& entity : entitiesWithRigidbody) {  
    if(entity.useGravity) {  
        entity.velocity.y -= gravityStrength * dt;  
    }  
}
```

Здесь:

- `gravityStrength` — коэффициент ускорения свободного падения (обычно около 9.81 м/с^2),
- `dt` — промежуток времени между предыдущими двумя кадрами.

2.2. Перемещение объектов

Следующий этап — обновить положение каждого объекта согласно изменению его скорости за прошедшее время.

```
entity.transform.position += entity.velocity * dt;
```

Таким образом, каждый кадр позиция объекта изменяется пропорционально его скорости и временному интервалу.

2.3. Проверка столкновений

Проверка на столкновения выполняется путем сравнения ограничивающих объемов (Axis-Aligned Bounding Boxes, AABB) каждого объекта с другими объектами, обладающими компонентом Collider.

Алгоритм прост:

- Для каждой пары объектов проверяются условия пересечения их ограничивающих коробок.
- Если обнаружено пересечение, выполняется коррекция положения и скоростей объектов.

Алгоритм проверки пересечений AABB:

```
bool Intersects(const AABB& box1, const AABB& box2) {  
    float dx = abs(box1.center.x - box2.center.x);  
    float dy = abs(box1.center.y - box2.center.y);  
    float overlapX = (box1.halfSize.x + box2.halfSize.x) - dx;  
    float overlapY = (box1.halfSize.y + box2.halfSize.y) - dy;  
    return overlapX > 0 && overlapY > 0;  
}
```

Где:

- center — центр ограничивающей коробки,
- halfSize — половина размеров ограничивающей коробки.

2.4. Разрешение конфликтов столкновений

Если два объекта пересеклись, необходимо исправить ситуацию следующим образом:

- Отталкиваем объекты друг от друга.
- Меняем направления и величины скоростей в зависимости от типа столкновения (упругое / неупругое).

```
if(intersectionDetected) {  
    ResolveCollision(entity1, entity2);  
}
```

Метод разрешения коллизий может выглядеть примерно так:

```

void ResolveCollision(Entity& ent1, Entity& ent2) {
    // Вычисляем глубину проникновения и направление разнесения
    vec2 normal = CalculateSeparationNormal(ent1.aabb, ent2.aabb);
    float penetrationDepth = CalculatePenetrationDepth(ent1.aabb, ent2.aabb);

    // Разносим объекты
    ent1.transform.position += normal * penetrationDepth * 0.5f;
    ent2.transform.position -= normal * penetrationDepth * 0.5f;

    // Исправляем скорости (упругое столкновение)
    ReflectVelocities(ent1, ent2, normal);
}

```

Итоги реализации:

По завершении этого шага у вас получится простая физическая система, которая способна:

- имитировать воздействие силы тяжести на игровые объекты;
- обновлять позиции объектов на основании текущих скоростей;
- проверять столкновения и исправлять ситуации, когда объекты пересекают друг друга.

Это создаст основу для более сложных механических взаимодействий, таких как удары, отражение снарядов, трение поверхностей и многое другое.

Шаг 3. Реализация InputManager

Управление вводом пользователя — одна из важнейших составляющих любого интерактивного приложения. В рамках построения простого, переносимого и надежного движка важно иметь единый входной слой, который позволит любым компонентам запрашивать состояние ввода (клавиатуры, мыши, геймпада и др.) простым способом, не зависящим от операционной системы или конкретной библиотеки графического интерфейса.

Цель данного шага — разработать класс InputManager, обеспечивающий удобный и единообразный доступ ко всей информации о действиях игрока (нажатия клавиш, перемещение мыши и др.). Важно сделать этот менеджер универсальным, удобным для расширения функционала и легко интегрируемым в существующий цикл обновлений игры.

Общий подход:

1. **Абстрактный уровень управления вводом.** Создаем единый интерфейс, через который любые модули смогут обращаться к состоянию клавиатуры/мыши.
2. **Унификация запросов.** Интерфейс делает работу с системой ввода независимой от конкретного устройства (клавиатура, мышь, контроллер и т.д.).
3. **Обеспечение производительности.** Все запросы состояния ввода выполняются синхронно и не замедляют игровой цикл.
4. **Расширяемость.** Позволяет создавать удобные карты привязок («actions») к физическим действиям игрока, таким как движение вперед, прыжок и т.д.

Архитектурный обзор:

3.1. Абстрактный интерфейс InputHandler

Интерфейс предоставляет общий способ чтения состояний устройств ввода, будь то клавиатура, мышь или другое устройство.

```
enum class KeyCode {
    KEY_UNKNOWN,
    KEY_A, KEY_B, /* остальные клавиши */
    MOUSE_LEFT, MOUSE_RIGHT, /* кнопки мыши */
    GAMEPAD_BUTTON_1, /* кнопки геймпадов */
    /* и т.д. */
};

// Базовый интерфейс для запроса состояния устройства ввода
class InputHandler {
public:
    virtual ~InputHandler() {};
    virtual bool IsKeyPressed(KeyCode key) const = 0;
    virtual bool IsMouseButtonPressed(int buttonIndex) const = 0;
    virtual glm::vec2 GetMousePosition() const = 0;
};
```

3.2. Основной класс InputManager

Отвечает за управление всеми источниками ввода и объединяет всю необходимую информацию о состоянии устройств ввода.

```
#include <map>
#include <glm/glm.hpp>

class InputManager final : public InputHandler {
public:
    bool IsKeyPressed(KeyCode key) const override {
        return m_KeyStates[key];
    }

    bool IsMouseButtonPressed(int buttonIndex) const override {
        return m_MouseButtons[buttonIndex];
    }

    glm::vec2 GetMousePosition() const override {
        return m_MousePosition;
    }

    // Метод для обновления состояния устройств ввода
    void UpdateState();

    // Привязываем физические действия к кнопкам ("MoveForward" -> W)
    void BindAction(const std::string& actionName, KeyCode keyCode);

    // Проверка активированного действия
```



```

bool IsActionActive(const std::string& actionName) const;

private:
    std::map<KeyCode, bool> m_KeyStates;
    std::map<int, bool> m_MouseButtons;
    glm::vec2 m_MousePosition;

    std::map<std::string, KeyCode> m_ActionsMap;
};

```

3.3. Опрос состояния ввода

Необходимо интегрировать обновление состояния ввода в игровой цикл. Лучше всего поместить проверку ввода в начало основного цикла перед этапом расчета физики и рендеринга сцены.

```

void GameLoop() {
    while (!ShouldQuit()) {
        inputManager.UpdateState(); // Получаем актуальные данные о нажатиях и положении
        // мыши
        physicsSystem.Update(dt); // Физическое моделирование
        renderSystem.RenderScene(); // Рендер сцены
    }
}

```

3.4. Поддержка действий

Привязывание физических действий к определенным клавишам облегчает дальнейшее расширение функциональности без повторения кода. Каждый модуль сможет запрашивать лишь статус нужного действия, не заботясь о специфике устройства ввода.

```

inputManager.BindAction("MoveForward", KeyCode::KEY_W);
inputManager.BindAction("Jump", KeyCode::KEY_SPACE);

if(inputManager.IsActionActive("MoveForward")) {
    player.MoveForward();
}

```

Расширение функциональности:

Для дальнейшего развития и поддержки различных платформ, возможно использование абстрактных классов для каждой категории устройств ввода (например, отдельный класс для работы с клавиатурой, отдельной класс для мыши и т.д.). Эти классы будут реализовывать общие методы типа Update() и обеспечивать совместимую структуру для различных библиотек (SDL, GLFW, SFML и др.).

Также можно добавить поддержку жестов, мультиплатформенность и отслеживание осей движения (например, аналоговых стиков на геймпадах).

Результат:

Получив готовый InputManager, мы получаем мощный инструмент для унификации обработки пользовательского ввода. Любой компонент теперь может легко проверять активность тех или иных команд («переместиться вперёд», «прыгнуть»), не задумываясь о технических особенностях оборудования и поддерживаемых операционных системах. Это значительно упростит разработку игры и сделает код более читаемым и понятным.

Шаг 4. Система событий коллизий

4.1. Создание EventBus

Global Event Bus представляет собой центральный объект, используемый для передачи уведомлений о событиях между различными частями приложения. Ключевое преимущество Event Bus заключается в возможности реализовать слабосвязанный дизайн, где компоненты могут отправлять и получать уведомления независимо друг от друга.

Пример как можно реализовать

```
class EventBus:
    def __init__(self):
        self.subscribers = {}

    def subscribe(self, event_type, subscriber):
        if event_type not in self.subscribers:
            self.subscribers[event_type] = []
        self.subscribers[event_type].append(subscriber)

    def publish(self, event):
        subscribers = self.subscribers.get(type(event).__name__, [])
        for sub in subscribers:
            sub.handle_event(event)
```

4.2. Определение класса CollisionEvent

Этот класс будет использоваться для отправки информации о произошедшем столкновении. Обычно такая структура включает минимум две сущности, участвующие в столкновении, и дополнительную метаданные (например, скорость удара, точка контакта и т.п.).

Пример как можно реализовать

```
class Entity:
    pass # Класс игровой сущности (упрощенно представлен)

class CollisionEvent:
    def __init__(self, entity_a, entity_b):
        self.entity_a = entity_a
        self.entity_b = entity_b
```

4.3. Реализация механизма обнаружения коллизий

В физическом менеджере (Physics System) обнаруживаются столкновения объектов. Как только произошло столкновение, система публикует событие через Event Bus.

Пример как можно реализовать

```
class PhysicsSystem:
    def check_collisions(self):
        entities = [] # список всех активных сущностей

        for i in range(len(entities)):
            for j in range(i + 1, len(entities)):
                e1 = entities[i]
                e2 = entities[j]

                # Условие проверки столкновения упрощено для примера
                if e1.position.distance_to(e2.position) <= (e1.radius + e2.radius):
                    collision_event = CollisionEvent(e1, e2)
                    event_bus.publish(collision_event)
```

4.4. Подписка на события

Подписчики регистрируются на получение конкретных типов событий и обрабатывают их соответствующим образом. Например, логгер коллизий записывает каждое столкновение в журнал.

Пример как можно реализовать

```
class LoggerSubscriber:
    def handle_event(self, event):
        print(f"Collision detected between {event.entity_a.name} and {event.entity_b.name}")

# Регистрация подписчиков
logger_subscriber = LoggerSubscriber()
event_bus.subscribe("CollisionEvent", logger_subscriber)
```

Примеры реакций на события:

Звуковые эффекты

Можно создать систему воспроизведения звуков при столкновениях, зарегистрировав обработчик, который проигрывает звуки ударов или взрывов:

```
class SoundPlayerSubscriber:
    def handle_event(self, event):
        sound.play_collision_sound() # Воспроизвести звуковой эффект
```

Изменение здоровья персонажа

В игре часто используется механика уменьшения здоровья при ударах. Это также реализуется через обработку событий:

```
class HealthManagerSubscriber:
    def handle_event(self, event):
        event.entity_a.health -= damage_value
        event.entity_b.health -= damage_value
```

Итоговая архитектура:

- **Простота интеграции:** новые механизмы легко подключаются через Event Bus.
- **Масштабируемость:** логика обработки коллизий отделяется от основной физики, позволяя добавлять больше функций без изменения ядра системы.
- **Удобство тестирования:** каждый модуль работает отдельно и тестируется изолированно, уменьшая риск ошибок.

Предложенная система событий обеспечивает удобную и эффективную инфраструктуру для взаимодействия между разными элементами игрового мира, позволяя быстро расширять функциональность игры.

Шаг 5. Debug-визуализация коллизий

Отладочная визуализация является важнейшим инструментом при разработке любых проектов, связанных с физикой и поведением объектов. Особенно полезна эта возможность при работе с алгоритмами столкновений и физикой движений. Идея состоит в том, чтобы выводить на экран прозрачные или пунктирные контуры, обозначающие границы коллайдеров. Это даст ясное представление о размерах объектов и причинах возникающих коллизий.

Основные цели:

1. Реализовать метод для отрисовки ограничительных рамок (DrawDebugAABB).
2. Организовать процедуру отрисовки рамок коллайдеров в режиме отладки после обработки столкновений.
3. Предоставить пользователю возможность быстрого переключения режима отладки с помощью горячих клавиш (например, клавиши F3).

5.1. Метод отрисовки границ коллайдеров (DrawDebugAABB)

Задача — создать функцию, которая рисует прозрачный или пунктирный контур вокруг коллайдера (граничащего объема AABB). Чтобы отрисовка была доступна вне зависимости от выбора графической библиотеки, лучше всего реализовать метод непосредственно в классе `Renderer` или подобном адаптере графического API (OpenGL/Vulkan/D3D).

Вот пример реализации для OpenGL:

```
#include <glm/vec4.hpp>
#include <glm/matrix.hpp>

class Renderer {
public:
    // Метод для отрисовки отладочного контура AABB
    void DrawDebugAABB(const AABB& aabb, const glm::vec4& color) {
        // Настройка прозрачности и толщины линий
        glEnable(GL_BLEND);
        glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
        glLineWidth(2.0f);
```

```

// Установка цвета рамки
glColor4fv(&color[0]);

// Конструируем матрицу преобразования
glm::mat4 modelMatrix = glm::translate(aabb.GetCenter());

// Переход в режим рисования линий
glBegin(GL_LINE_LOOP);
// Координаты вершин по сторонам ограничивающего блока
glVertex3f(-aabb.HalfExtents().x, -aabb.HalfExtents().y, -aabb.HalfExtents().z);
glVertex3f(+aabb.HalfExtents().x, -aabb.HalfExtents().y, -aabb.HalfExtents().z);
glVertex3f(+aabb.HalfExtents().x, +aabb.HalfExtents().y, -aabb.HalfExtents().z);
glVertex3f(-aabb.HalfExtents().x, +aabb.HalfExtents().y, -aabb.HalfExtents().z);
// Остальные стороны аналогичным образом...
glEnd();

// Возвращаемся к стандартным настройкам OpenGL
glDisable(GL_BLEND);
}
};

```

5.2. Организация вывода коллайдеров в режиме отладки

Необходимо настроить вызов метода отрисовки после того, как завершится обработка столкновений. Для этого удобно ввести дополнительный этап в цикле обновления игры, ответственный за отладочные операции.

Например, добавьте новый метод в вашу систему физики (PhysicsSystem) или создайте специальную отладочную систему (DebugRenderSystem):

```

class PhysicsSystem {
public:
    void Update(float deltaTime) {
        ProcessCollisions(deltaTime); // Обрабатываем столкновения
        if(DebugRenderingEnabled()) {
            RenderDebugColliders(); // Выводим рамки коллайдеров
        }
    }

private:
    void RenderDebugColliders() {
        for(const auto& entity : entitiesWithColliders) {
            renderer.DrawDebugAABB(entity->collider.aabb, glm::vec4(1.0f, 0.0f, 0.0f, 0.5f));
        }
    }
};

```

Метод RenderDebugColliders перебирает все активные коллайдеры и отправляет их на отрисовку.

5.3. Переключение режима отладки

Настроим включение и отключение отладочного режима с помощью удобной горячей клавиши (например, F3).

Используйте ваш InputManager для захвата события нажатия клавиши:

```
class InputManager {
public:
    void OnKeyDown(KeyCode key) {
        if(key == KeyCode::KEY_F3) {
            toggleDebugRendering();
        }
    }

private:
    void toggleDebugRendering() {
        setDebugRendering(!isDebugRenderingEnabled()); // Инверсия текущего флага
    }
};
```

В результате при нажатии клавиши F3 включается или выключается режим отладки, соответственно показывая или скрывая ограничения коллайдеров.

Флаги для отладки можно хранить прямо в основном приложении или системе рендеринга.

Итог реализации:

Завершив этот шаг, вы получите полезный инструмент для отладки и анализа ваших физических алгоритмов. Теперь вы сможете наблюдать реальные размеры коллайдеров, выявляя несоответствия или проблемы в расположении объектов. Это повысит эффективность вашего рабочего процесса и сократит количество багов, связанных с ошибочным позиционированием или неправильной работой физики.

Шаг 6. Управление 3D-камерой

Правильно спроектированная и реализованная система управления камерой — это основа комфортного восприятия и взаимодействия с игровым миром. Камера выступает глазами игрока, определяя его опыт и погружённость в происходящее. Грамотно настроенная камера позволяет:

- **Свободно исследовать сцену.** Игрок может осмотреть объекты с разных ракурсов, изучить окружающую обстановку и принять тактические решения.
- **Оценить расстояния и масштабы.** Хорошее управление камерой помогает точно определять расстояние до объектов, планировать траекторию движения и избегать препятствий.
- **Повысить вовлеченность и интерес.** Плавные и отзывчивые движения камеры усиливают ощущение присутствия и улучшают общее впечатление от игры.

Основные цели:

1. Добавить компонент Camera, содержащий необходимые данные для представления позиции и направления взгляда камеры.

2. Создать систему CameraSystem, которая считывает пользовательские команды через InputManager и применяет изменения положения и вращения камеры.
3. Передавать параметры камеры (view matrix, projection matrix) в RenderAdapter.

6.1. Компонент Camera

Камера представляет собой сущность, обладающую положением и направлением, позволяющими просматривать сцену. Её основные поля включают:

```
struct CameraComponent {  
    glm::vec3 position;    // Позиция камеры в мировом пространстве  
    glm::vec3 forward;    // Направление вперёд  
    glm::vec3 up;         // Верхний вектор (чаще всего (0, 1, 0))  
    float fov;            // Угол обзора  
    float aspectRatio;    // Соотношение сторон экрана  
    float nearPlane;      // Ближняя граница отсечения  
    float farPlane;       // Дальняя граница отсечения  
};
```

Эти данные используются для вычисления матриц вида и перспективы, необходимых для правильного рендеринга сцены.

6.2. Система CameraSystem

Система отвечает за интерпретацию команд пользователя (перемещение вперед/назад, стрейф вправо/влево, подъем вверх/вниз, повороты камеры) и последующее обновление состояния компонента Camera. Она полагается на InputManager для отслеживания состояний клавиатуры и мыши.

Пример реализации:

```
class CameraSystem {  
public:  
    void Update(float deltaTime) {  
        // Получаем текущее состояние ввода  
        bool moveForward = inputManager.IsKeyPressed(KeyCode::W);  
        bool moveBackward = inputManager.IsKeyPressed(KeyCode::S);  
        bool strafeLeft = inputManager.IsKeyPressed(KeyCode::A);  
        bool strafeRight = inputManager.IsKeyPressed(KeyCode::D);  
        bool mouseMoved = inputManager.MouseDelta().length() > 0.f;  
  
        // Обработка перемещения  
        if(moveForward || moveBackward || strafeLeft || strafeRight) {  
            HandleMovement(deltaTime);  
        }  
  
        // Обработка поворота камеры
```

```

    if(mouseMoved) {
        HandleRotation(deltaTime);
    }

    // Рассчёт новых значений view и projection matrices
    CalculateViewProjectionMatrices();
}

private:
void HandleMovement(float deltaTime) {
    glm::vec3 direction = camera.forward;
    float speed = 5.0f * deltaTime;

    if(inputManager.IsKeyPressed(KeyCode::W)) { // Вперед
        camera.position += direction * speed;
    }
    else if(inputManager.IsKeyPressed(KeyCode::S)) { // Назад
        camera.position -= direction * speed;
    }

    if(inputManager.IsKeyPressed(KeyCode::A)) { // Стрейф влево
        camera.position -= glm::normalize(glm::cross(direction, camera.up)) * speed;
    }
    else if(inputManager.IsKeyPressed(KeyCode::D)) { // Стрейф вправо
        camera.position += glm::normalize(glm::cross(direction, camera.up)) * speed;
    }
}

void HandleRotation(float deltaTime) {
    glm::vec2 mouseDelta = inputManager.MouseDelta();
    float sensitivity = 0.1f;

    glm::quat rotationY = glm::angleAxis(sensitivity * mouseDelta.x, glm::vec3(0, 1, 0)); //
Поворот по горизонтали
    glm::quat rotationX = glm::angleAxis(sensitivity * mouseDelta.y, glm::vec3(1, 0, 0)); //
Наклон вверх-вниз

    camera.forward = glm::rotate(rotationY, glm::rotate(rotationX, camera.forward));
}

void CalculateViewProjectionMatrices() {
    // Вычисляем view-матрицу
    glm::mat4 viewMatrix = glm::lookAt(camera.position, camera.position + camera.forward,
camera.up);

    // Вычисляем projection-матрицу
    glm::mat4 projMatrix = glm::perspective(glm::radians(camera.fov), camera.aspectRatio,
camera.nearPlane, camera.farPlane);

    renderAdapter.SetViewProjection(viewMatrix, projMatrix);
}
};

```


6.3. Использование InputManager

Чтобы система работала корректно, необходимо передавать ей обработанные сигналы от клавиатуры и мыши. Здесь пригодится созданный ранее InputManager, который сможет перехватывать такие события, как нажатие клавиш и движения мыши.

Пример использования в игровом цикле:

```
void GameLoop() {
    while(application.isRunning()) {
        inputManager.Update(); // Опрос входных данных
        physicsSystem.Update(deltaTime); // Обновление физики
        cameraSystem.Update(deltaTime); // Обновление камеры
        renderSystem.Render(); // Рендеринг кадра
    }
}
```

Результат: Игрок может свободно перемещаться по сцене и осматривать физические объекты.

Шаг 7. Демонстрационная сцена

Демонстрационная сцена служит наглядным примером работоспособности созданных вами систем физики, ввода и отладки. Цель этапа — показать интеграцию различных элементов, продемонстрировав физическое поведение объектов в трехмерном пространстве, управляемость игроком и стабильность системы.

7.1. Подготовка сцены

Подготовка базовых элементов сцены:

1. **Поверхность:** Представьте землю в виде плоского квадрата с размерами, достаточными для размещения объектов.
2. **Объекты с физическим поведением:**
 - Несколько статичных объектов (без массы, например, земля).
 - Один или несколько динамичных объектов (кубы, сферы), оснащенных компонентами Rigidbody и Collider. Эти объекты должны подчиняться законам физики, включая падение под воздействием силы тяжести и реакцию на столкновения.
3. **Управляемый объект:** Можно выбрать один из динамичных объектов, добавив ему способность реагировать на команды игрока (использование компонента Rigidbody совместно с системой управления).
4. **Освещение:** Хотя освещение не связано напрямую с физической симуляцией, оно улучшает восприятие сцены и делает её более привлекательной.

7.2. Постановка эксперимента

Основная задача демонстрации заключается в следующем:

- Показать корректное функционирование физики: тела падают на поверхность земли и останавливаются благодаря обработке столкновений.
- Продемонстрировать работу управляемого объекта: игрок управляет одним объектом, заставляя его двигаться и сталкиваться с другими элементами сцены.
- Проиллюстрировать использование отладочной визуализации: видимые границы коллайдеров позволяют убедиться, что столкновения происходят правильно.

7.3. Примеры объектов и их поведения

Рассмотрим простейшую ситуацию с двумя типами объектов:

1. Кубики на плоскости:

- Три-четыре кубика размещены на поверхности уровня, одни статичные, другие динамические.
- Динамический кубик падает на землю и остаётся там благодаря правильной реакции на столкновения.

2. Шары:

- Пара шарообразных объектов, помещённых рядом с кубиками, демонстрирует различие в поведении (например, шарики имеют меньшую массу и могут катиться по земле).

3. Управляемый объект:

- Игрок контролирует один из кубиков посредством кнопок WASD (движение вперёд, назад, влево, вправо) и пробела (прыжок).

7.4. Последовательность действий

1. Создание сцены:

- Разместите подготовленные объекты (земля, кубики, шары) в редакторе.
- Примените к каждому объекту соответствующие компоненты: Rigidbody, Collider, MeshRenderer.

2. Физическое поведение:

- Активируйте применение гравитации на объектах, настроив компоненты Rigidbody.
- Проверьте, что объекты адекватно реагируют на столкновения и замедляются при соприкосновении с поверхностью.

3. Управление:

- Установите связь между командой управления (WASD, прыжок) и изменением свойств объекта (позиции, скорости).
- Используйте InputManager для обработки ввода и отправки соответствующих сигналов в систему.

4. Отладка и проверка:

- Включите режим отладки, используя клавишу F3, чтобы увидеть границы коллайдеров и убедиться, что всё функционирует верно.
- Запустите игру и убедитесь, что управляемый объект слушается команд, а остальные элементы ведут себя предсказуемо.

Результат: Полноценная демонстрация физики и ввода в 3D-пространстве.

ЗАДАНИЕ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

1. **Разные типы коллайдеров:** добавить поддержку сферических коллайдеров и их взаимодействие с AABB.
2. **Трение и упругость:** реализовать параметры bounciness (коэффициент упругости) и friction (трение) в компоненте Collider .
3. **Триггеры:** добавить возможность помечать коллайдер как триггер (не создаёт физического отталкивания, но генерирует событие TriggerEvent).
4. **Пространственное разбиение:** оптимизировать проверку коллизий с помощью Grid или Quadtree.

РЕКОМЕНДАЦИИ ПО ВЫПОЛНЕНИЮ РАБОТЫ

1. Фундаментальные принципы разработки

1.1. Фиксированный шаг физики

Для достижения стабильного и предсказуемого поведения физической системы крайне важно соблюдать постоянный шаг обновления физики. Рекомендуемый интервал — **1/60 секунды** (примерно 16.67 миллисекунд). Даже если частота кадров рендеринга варьируется, физика должна обновляться с постоянной скоростью. Это предотвратит артефакты и нестабильность поведения объектов.

1.2. Порядок обновления

Соблюдайте чёткий порядок обновления систем:

1. **Опрос ввода** — считывание данных с устройств ввода.
2. **Обновление физики** — расчет гравитации, коллизий и движений объектов.
3. **Рендеринг** — отрисовка сцены с учётом последних изменений.

Такой порядок минимизирует задержки и обеспечивает плавность взаимодействия.

2. Эффективная отладка и визуализация

2.1. Debug-рендеринг

Отладочная визуализация — незаменимый инструмент при разработке физики. Обязательно реализуйте возможность отрисовки коллайдеров (AABB, сферы) в мировых координатах. Преобразование AABB должно учитывать трансформу сущности и смещение коллайдера относительно неё.

Пример реализации:

```
void RenderAdapter::drawDebugAABB(const AABB& aabb, const glm::vec4& color) {  
    // Преобразование AABB в мировые координаты  
    glm::mat4 transformMatrix = entity.transform.matrix * aabb.offsetMatrix;  
    // Отрисовка каркаса AABB  
    drawWireframe(transformMatrix, color);  
}
```

2.2. Визуализация коллизий

Визуализация коллайдеров и нормалей столкновений значительно облегчит диагностику проблем с физикой. Активируйте отладочную визуализацию через горячую клавишу (например, F3), чтобы оперативно включать и отключать её.

3. Надёжная реализация событий

3.1. Минималистичная реализация событий

Используйте легковесную реализацию диспетчера событий, избегая дорогостоящих динамических аллокаций в критических участках кода. Простой и быстрый способ — комбинация `std::function` и контейнера-списка подписчиков.

Пример реализации:

```
class EventDispatcher {  
public:  
    void addListener(const std::string& eventName, std::function<void()> listener) {  
        listeners[eventName].emplace_back(listener);  
    }  
  
    void dispatchEvent(const std::string& eventName) {  
        auto it = listeners.find(eventName);  
        if (it != listeners.end()) {  
            for (const auto& listener : it->second) {  
                listener();  
            }  
        }  
    }  
  
private:  
    std::unordered_map<std::string, std::vector<std::function<void()>>> listeners;  
};
```

4. Стратегия поэтапного тестирования

4.1. Постепенное наращивание сложности

Выполняйте работу постепенно, проводя тестирование после каждого важного этапа:

1. Начните с **простого случая гравитации** без учёта коллизий (падение объектов на неподвижную поверхность).
2. Добавьте **детектор коллизий AABB-AABB** и проверьте корректность срабатывания.

3. Реализуйте **разрешение коллизий** (отталкивание объектов).
4. Постепенно увеличивайте сложность тестов, добавляя наклонные поверхности, подвижные объекты и триггеры.

4.2. Пошаговое тестирование

После каждого шага компилируйте и проверяйте работоспособность:

- Проверьте, что объекты стабильно падают под действием гравитации.
- Убедитесь, что коллизии корректно обнаруживаются и разрешаются.
- Оцените плавность и точность управления камерой.

5. Оптимизация и математика

5.1. Библиотека математики

Используйте библиотеку **GLM** для работы с векторами и матрицами. Это значительно упростит и ускорит разработку.

5.2. Реализация проверок коллизий

Реализуйте функции пересечения AABB самостоятельно. Несмотря на кажущуюся сложность, это довольно простая операция, сводящаяся к проверке перекрытия отрезков на каждой оси.

Пример псевдокода:

```
bool intersectAABB(const AABB& a, const AABB& b) {  
    return !(a.max.x < b.min.x || a.min.x > b.max.x ||  
            a.max.y < b.min.y || a.min.y > b.max.y ||  
            a.max.z < b.min.z || a.min.z > b.max.z);  
}
```

5.3. Производительность

На начальном этапе допустима проверка коллизий «в лоб» за $O(n^2)$. Для небольших сцен (до 50 объектов) это приемлемо. Позже можно перейти к пространственному разбиению (Grid, QuadTree, Octree).

6. Интеграция с ECS

6.1. Поиск сущностей с коллайдерами

Для эффективного поиска пар сущностей с компонентами Transform и BoxCollider используйте встроенные возможности ECS (например, Registry View):

```
auto view = registry.view<Transform, BoxCollider>();  
for (auto entity : view) {  
    // Проверка коллизий  
}
```

7. Система событий и сервис-локатор

7.1. Глобальный диспетчер событий

Для простоты сделайте диспетчер событий глобально доступным через сервис-локатор. Это упростит подключение и использование событий в различных частях движка.

Пример реализации:

```
class ServiceLocator {
public:
    static EventDispatcher& getEventDispatcher() {
        static EventDispatcher dispatcher;
        return dispatcher;
    }
};
```

Следуя изложенным рекомендациям, вы обеспечите качественную и стабильную реализацию физической системы и системы ввода. Помните, что постепенное тестирование, правильная организация кода и внимательная отладка — залог успеха при разработке сложных подсистем.

**ПОМНИТЕ, У КАЖДОЙ ЗАДАЧИ ЕСТЬ НЕСКОЛЬКО ВАРИАНТОВ РЕШЕНИЯ,
ПРЕДЛОЖЕННЫЙ ВАРИАНТ НА ЯВЛЯЕТСЯ АКСИОМОЙ!!!**

ВАРИАНТ ПОСЛЕДОВАТЕЛЬНОСТИ ВЫПОЛНЕНИЯ

1. Создание компонентов Rigidbody и Collider.
2. Реализация PhysicsSystem (гравитация + обновление позиций).
3. Добавление обнаружения и разрешения коллизий AABB.
4. Создание InputManager.
5. Интеграция событий коллизий.
6. Реализация debug-отрисовки AABB.
7. Добавление управления камерой.
8. Сборка демо-сцены и тестирование.

ФОРМА И СОДЕРЖАНИЕ ОТЧЕТА

1. **Титульный лист** (ФИО, группа, название работы).
2. **Введение** (цель, задачи).
3. **Архитектура решения:**

Диаграмма классов физических компонентов и систем.

Описание взаимодействия с ECS и RenderAdapter.

4. **Реализация ключевых модулей:**

Детектор коллизий AABB (листинг с пояснениями).

Применение гравитации.

InputManager и маппинг действий.

Debug-визуализация.

5. Результаты:

Скриншоты сцены с debug-отрисовкой коллайдеров.

Скриншоты консоли с событиями коллизий.

Видео/гифка с движущимися объектами (опционально).

6. Трудности и их решение.

7. Заключение (выводы, возможные улучшения).

ПРИЛОЖЕНИЯ К ОТЧЕТУ

Ссылка на Git-репозиторий с полным кодом проекта.

CMakeLists.txt или файлы проекта.

Конфигурационные файлы (если использовались).

Видео-демонстрация работы (опционально).

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. В чём разница между OnCollision и OnTrigger в контексте физики?
2. Почему для физики предпочтительнее фиксированный шаг обновления?
3. Как рассчитать силу гравитации, чтобы объекты падали с реалистичным ускорением?
4. Какие существуют стратегии разрешения коллизий (position correction, impulse-based)?
5. Как организовать маппинг действий (например, «прыжок») на разные клавиши?
6. Для чего нужно пространственное разбиение при обнаружении коллизий?
7. Как преобразовать локальный AABB коллайдера в мировое пространство для отрисовки?
8. Какие недостатки имеет AABB по сравнению с OBB (Oriented Bounding Box)?

РЕКОМЕНДУЕМАЯ ЛИТЕРАТУРА

1. **Game Engine Architecture** – Jason Gregory (глава "Physics and Collision Detection").
2. **Real-Time Collision Detection** – Christer Ericson (классическая книга по алгоритмам коллизий).

3. **Документация Unity Physics** — полезно для понимания практических аспектов физики в играх .
4. **Серия статей "Game Physics"** на [Gamedev.net](https://gamedev.net).
5. **"Паттерны проектирования в игровой разработке"** — Роберт Нистром (главы про команду и событийность).

Примечание:

Данная работа объединяет две важнейшие подсистемы движка — физику и ввод. Качественная реализация на этом этапе станет основой для создания полноценного геймплея в итоговом проекте. Уделите особое внимание стабильности физики при разных значениях `deltaTime` и удобству использования `InputManager`.